Universidad de las Fuerzas Armadas - ESPE

Nombre: Jhostyn Adriano Paca Pérez

NRC: 36460 Materia: Fundamentos de programación

Fecha: 04-11-2026

Ejercicio 1.1 Media de dos valores

• Pseudocódigo

```
Algoritmo Media-de-dos-numeros
    Definir x, y, media Como Real;
    Leer x;
    Leer y;
    media ← (x+y)/2;
    Escribir media;
Fin Algoritmo
```

• Código fuente

```
// Ejercicio 1 Cálculo de la media aritmética de dos números
// Autor : Jhostyn Paca
// NRC : 36460
#include <stdio.h>
int main (){
    // Variables para almacenar los dós números ingresados y el resultado
    // de la media
    float a, b, c;
    // Solicitar el primer número al usuario
    printf ("Ingrese el valor del primer número \n");
    scanf ("%f ,&a);
```

```
// Validar que el número esté en el rango permitido [0 , 20]
while (a < 0 || a > 20 ){
    printf ("Ingrese un valor positivo en el rango de 0 a 20\n");
    scanf ("%f" &a);
}
// Solicitar el segundo número al usuario
printf ("Ingrese el valor del segundo número\n");
scanf ("%f", &b);
// Validar que el número este en el rango permitido [0 , 20]
while ( b < 0 || b > 20) {
    printf ("Ingrese un valor positivo en el rango de 0 a 20\n");
    scanf ("%f", &b)
}
// Calcular la media aritmética de los dos números
c = (a + b) / 2;
// Mostrar el resultado al usuario
printf ("La media de ambos números es: %f", c);
return 0;
}
```

Ejercicio 7.2 Valor absoluto de x al cubo

• Pseudo código

```
Algoritmo valor_absoluto_al_cubo
    Definir x, c Como Real
    Leer x
    Si x<0 Entonces
        c ← (x*x*x)*(-1)
    Sino
        c ← x*x*x;
    Fin Si
    Escribir c;
Fin Algoritmo
```

• Código fuente

```
// Ejercicio 2 ¡Calcular el valor absoluto de un número elevado al cubo
// Jhostyn Adriano Poca Pérez
// NRC:36460
#include<stdio.h>
int main(){
    // Variable de entrada y variable para almacenar el resultado
    float a,b;
    // Solicitar el número al usuario
    printf("Ingrese el valor para sacar su valor absoluto y sacarlo al cubo\n");
    scanf("%f",&a);
    // Si el número es negativo, se cambia el signo de su cubo
    // para obtener el valor absoluto del resultado
    if(a<0){
```

```
        b = (-1) * (a * a * a);
    }
    else {
        // Si el número es positivo o cero, el cubo ya es positivo
        b = a * a * a;
    }
    // Mostrar resultado obtenido
    printf ("El valor del valor absoluto de %.2f al cubo es: %.2f", a, b);
    return 0;
}
```

Ejercicio 1.3 Divisible

• Pseudocódigo

```
Algoritmo divisibilidad
    Definir x, y, c Como Real
    Leer x, y;
    Si y == 0;
        Escribir Error
    Sino
        Si x / y = 0
            Escribir x "es divisible para" y;
        Sino
            Escribir x "no es divisible para" y;
        FinSi
        c = x / y;
        Escribir "El resultado de" x "/" y "es:" c
    FinSi
FinAlgoritmo
```

• Código Fuente

```
// Ejercicio 3
// Divisible
// NRC: 36460
int main() {
    // Variables para almacenar los dos números y el residuo de la división
    int a, b, c;
    // Solicitar el primer número
    printf("Ingrese primer valor \n");
```

```
scanf ("%d", &a);
// Solicitar el segundo número
printf ("Ingrese el segundo valor \n ");
scanf ("%d", &b);
// Calcular el residuo de la división
c = a %b;
// Verificar el residuo de la división
c = a %b;
if (c==0){
    // Si el residuo es cero, el primer número es divisible para el
       segundo
    printf ("%d es divisible para %d", a, b);
}
else {
    // Si el residuo es diferente de cero, no es divisible
    printf ("%d no es divisible para %d", a, b);
}
return 0;
}
```

Ejercicio 7.4 Intervalo

• Pseudocódigo

```
Algoritmo Intervalo
    Definir x Como entero
    Leer x
    Si 0≤ x Y x ≤10 entonces
        Escribir x "Pertenece al intervalo de [0 , 10]";
    Sino
        Escribir
    Fin Si    x "No pertenece al intervalo de [0, 10]";
Fin Algoritmo
```

• Código Fuente

```
// Ejercicio 4 ¿Verificar que un número pertenezca al intervalo [0, 10]
// Jhostyn Adriano Paca Pérez
// NRC: 36460
# include<studio.h>
int main (){
    // Variable para almacenar el número ingresado por el usuario
    int a ;
    // Solicitar un número dentro del intervalo permitido
    printf ("Ingrese un número que se encuentre en el intervalo de [0, 10]\n");
    scanf ("%d", &a );
    // Validar que el número se encuentre en el intervalo de [0 , 10]
    while (a <0|| a > 10){
        // Solicitar nuevamente el número si se encuentra fuera de rango
        printf ("ERROR : Ingrese un número que se encuentre en el
```

```
    intervalo de [0, 10] \n");
    scanf("%d", &a)
}
// Confirma que el número pertenece al intervalo
printf("%d existe en el intervalo de [0, 10]", a);
return 0;
}
```

Ejercicio 1.5

•

Ejercicio 1.5: Conversion de unidades de tiempo

• Pseudocódigo

```
Algoritmo Conversión_de_unidades_de_tiempo
    Definir c, dias, horas, minutos, segundos Como Entero;
    Leer c;
    Si c<0 Entonces
        Escribir "Error";
    Sino
        dias ← trunc (c/86400);
        c ← c %86400;
    Escribir "El monto tiene", dias, "dias";
    horas ← trunc (c/3600);
    c ← c %3600;
    Escribir horas, "horas";
    minutos ← trunc (c/60);
    c ← c %60
    Escribir minutos, "minutos";
    Escribir "y", c, "segundos";
    Fin Si
Fin Algoritmo
```

• Código fuente

```
// Ejercicio 5 Conversión de una cantidad total de segundos a días, horas, minutos y segundos
// Jhostyn Adriano Paca Pérez
// NRC 36460
# include <stdio.h>
int main (){
```

```
// Variables para almacenar el tiempo descompuesto
int c, d, h, m, s;
// Solicitar el tiempo total en segundos
printf("Ingrese el total de segundos en el evento\n");
scanf("%d", &c);
// Validar que la cantidad de segundos sea positiva
while (c < 0) {
    printf("ERROR Ingrese un número positivo\n");
    scanf("%d", &c);
}
// Calcular la cantidad de días completos
d = c / 86400;
c = c % 86400;
// Calcular la cantidad de horas restantes
h = c / 3600;
c = c % 3600;
// Calcular la cantidad de segundos restantes y minutos
m = c / 60;
s = s % 60;
// Mostrar la conversión del tiempo
printf("El evento tiene un total de %d días, %d horas, %d minutos
y %d segundos", d, h, m, s);
return 0;
}
```

Ejercicio 1.6: Máximo, mínimo y media de alturas

- Pseudo código

```
Algoritmo maxmin y mínimo
    Definir n,i Como Entero;
    Definir a,s,media Como Real;
    Leer n;
    Si n<0 entonces;
        Escribir "Error";
    Sino
        Para i<-1 Hasta n Con Paso 1 Hacer
            Escribir "Escriba la altura #",i;
            Leer a;
            s<-a+s;
        Fin Para
        media<-s/n;
        Escribir media;
    FinSi
Fin Algoritmo
```

- Código Fuente

```
// Ejercicio 6: Determina la altura máxima, mínima y media de un conjunto de edificios
// Jhostyn Adriana Parra Pérez
// NRC: 36460
#include <studio.h>
int main(){
    // Variables para el contador, cantidad de edificios, altura ingresada, altura máxima, altura mínima y media
    int i,n;
    float a,MAX,MIN,m;
```

```
// Solicitar la cantidad de edificios a analizar
printf("Ingrese la cantidad de edificios a analizar\n");
scanf("%d", &n);
// Validar que la cantidad de edificios sea positiva
while (n <= 0) {
    printf("ERROR: La cantidad de edificios DEBE ser positiva\n");
    scanf("%d", &n);
}
// Ingrese la cantidad de altura del primer edificio
printf("Ingrese la cantidad de altura del edificio 1:\n");
scanf("%f", &a);
// Inicializar los valores máximos, mínimos y acumulador con la primera altura ingresada
MAX = a;
MIN = a;
m = a;
// Leer las alturas de los edificios restantes
for (i = 2; i <= n; i++) {
    printf("Ingrese la altura del edificio %d:\n", i);
    scanf("%f", &a);
// Actualizar la altura máxima encontrada
if (a > MAX) {
    MAX = a;
}
// Actualizar la altura mínima encontrada
if (a < MIN) {
    MIN = a;
}
// Acumular las alturas para calcular los medios
m = m + a
}
// Calcular la media de las alturas ingresadas
m = m / n;
// Mostrar los resultados obtenidos
printf("La altura Maxima de los %d edificios es %.2f, \n la altura minima es %.2f \n la media de todos los edificios ingresados es %.2f", n, MAX, MIN, m);
return 0;
}
```

Ejercicio 7.7: Raiz cuadrada entera

• Pseudo código

```
Algoritmo Raiz_cuadrado_entero
    Definir x, i, z;
    Leer x;
    Si x>0 entonces
    i<-1;
    Mientras i*1 <=x Hacer
      i<-i+1;
    Fin Mientras
    z<-i-1
    Escribir "la raiz cuadrada entera por defecto de", x, "es", z
  Sino
    Escribir "ERROR";
  Fin Si
Fin Algoritmo
```

• Código Fuente

```
// Ejercicio 7: Calcular la raiz cuadrada entera de un número positivo
// DNI: 36490
// Jhonlyn Adriano Paca Perez

#include <stdio.h>
int main (){

    // Variables para almacenar el número ingresado y el resultado de la raiz cuadrada entera
    int a,b,c;

    // Solicitar el número al usuario
    printf("Ingrese un número para sacar su raiz cuadrada entera\n");
    scanf("%d", &a);

    // Validar que el número sea mayor que 0
    while (a<=0){
```

ESTILO

```
        printf("Error, Ingrese un numero mayor a 0 \n");
        scanf("%d", &a);
    }

    // Inicializar el contador que permitira encontrar la mayor raiz entera cuyo cuadrado supere al número
    b = 0

    // Incrementar el contador mientras su cuadrado sea menor o igual al número ingresado
    while (b * b <= a) {
        b++;
    }

    // Retroceder una unidad para obtener la raiz entera correcta
    b--;

    // Mostrar el resultado
    printf("La raiz cuadrada entera de %d es %d", a, b);

    return 0;
}
```

Ejercicio 7.8: Número primo

• Pseudocódigo

```
Algoritmo número_primo
    Definir x, n Como entero
    Leer x;
    Si x<0;
        Escribir Error;
    Sino
        Para i←1 Hasta x Con Paso 1 Hacer
            Si x%i==0;
                n←n+1;
            FinSi
        FinPara
        Si n==2 Entonces
            Escribir "El número", x, "es primo";
        Sino
            Escribir "El número", x, "no es primo";
        FinSi
    FinSi
FinAlgoritmo
```

• Código fuente

```
// Ejercicio 8: Determinar si un número es primo
// Jhostyn Adriano Poca Pérez
// URC: 36460

#include<studio.h>
int main(){
    //Variables para almacenar el número ingresado, la cantidad de divisores encontrados
      y el contador
```

```
int x, n = 0, i;
// Solicitar el número al usuario
printf("Ingrese un número para verificar si es primo o no \n");
scanf("%d", &x);

// Validar que el número sea mayor que cero
while (x <= 0){
    printf("ERROR: Ingrese un valor positivo mayor a 0\n");
    scanf("%d", &x);
}

// Recorrer todos los números desde 1 hasta el valor ingresado para contar cuantos divisores tiene
for (i = 1; i <= x; i++){
    // Verificar si el número es divisible exactamente por i
    if (x % i == 0){
        n++;
    }
}

// Un número es primo si tiene exactamente 2 divisores: el 1 y el mismo
if (n == 2){
    printf("El número %d es primo", x);
}
else{
    printf("El número %d no es primo", x);
}

return 0;
}
```

Ejercicio 1.9 Números primos comprendidos entre dos números dados

- Pseudo código

```
Algoritmo Numeros_primos_comprendidos_entre_dos_numeros_dados
    Definir n,x,i,j,dom1,dom2 Como Entero;
    Definir Lista_primos Como cadena;
    Escribir "Da el dominio: ";
    Leer dom1, dom2;
    n←0
    Si dom1≤0 O dom2≤0 Entonces
        Escribir "Error";
    Sino
        Para i dom1 Hasta dom2 Con Paso 1 Hacer
            x←0
            Para j Hasta i Con Paso 1 Hacer
                Si i%j == 0;
                    x←x+1;
                Fin Si
            Fin Para
            Si x==2 Entonces
                n=n+1;
                Lista_primos←Lista_primos+ConvertirTexto(i)+" ";
            Fin Si
        Fin Para
        Escribir "Existen" n "números primos en ese dominio";
        Escribir "los números primos en el dominio de [", dom1, ",", dom2, "] son:";
            Lista_primos
    Fin Si
Fin Algoritmo
```

- Código Fuente

```
// Ejercicio 9, Encontrar los números primos dentro de un intervalo dado
// Thorton Adriano Paca Pérez
// UAC: 36460

#include <stdio.h>
int main(){
    // Variables para contar números primos, divisores y recorrer el intervalo
    int n, x, i, j, dom1, dom2;
    // Solicitar los límites de los intervalos
    printf("Ingrese desde donde empezaran a contarse los numeros primos\n");
    scanf("%d", &dom1);

    printf("Ingrese donde terminara el intervalo\n");
    scanf("%d", &dom2);
```

```
// Inicializar el contador de números primos encontrados
n = 0

// Validar que el intervalo sea correcto
while ( dom1 < 0 || dom2 < dom1 ) {
    printf("ERROR\n");
    printf("Ingrese desde donde se inicia los números primos\n");
    scanf("%d", &dom1);
    printf("Ingrese donde termina el intervalo\n");
    scanf("%d", &dom2);
}

// Mostrar los números primos encontrados del intervalo
for ( i = dom1 ; i <= dom2 ; i++ ) {
    // Reiniciar el contador para todos los números del intervalo
    x = 0;

    // Contar cuantos divisores tiene el número actual
    for ( j = 1 ; j <= i ; j++ ) {
        if ( i % j == 0 ) {
            x++;
        }
    }
    // Un número es primo si tiene exactamente 2 divisores
    if ( x == 2 ) {
        // Incrementar el contador de números primos encontrados
        n = n + 1;
        // Mostrar el número primo
        printf("%d", i);
    }
}

// Mostrar la cantidad total de números primos encontrados
printf("\n Existen %d cantidad de números primos en dominio de %d hasta %d\n", n, dom1, dom2);

return 0;
}
```

Ejercicio 10: Factorial

- Pseudocódigo

```
Algoritmo Factorial
    Definir n, nF, i Como Entero
    Leer n;
    nF ← 1;
    Para i ← 1 Con Paso 1 Hasta n Hacer
        nF ← nF * i
    Fin Para
    Escribir nF;
Fin Algoritmo
```

- Código Fuente

```
// Ejercicio 10: Calcular el factorial de un número
// Jhostyn Adriano Poca Pérez
// UAC; 36460
#include <stdio.h>
int main(){
    // Variables para el contador, el número ingresado y el resultado del factorial
    int i,n,nF;
    // Solicitar el número al usuario
    printf("Ingrese el valor para sacar su factorial\n");
    scanf("%d",&n);
    // Inicializar el factorial en 1
    nF = 1;
    // Multiplicar sucesivamente dos números desde 1 hasta n
    for(i = 1; i <= n; i++){
        // Acumular el factorial por sus productos
        nF=nF * i;
    }
```

```
// Mostrar el resultado obtenido
printf("El factorial de %d es %d", n, nf);
return 0;
}
```

Ejercicio 1.11: Serie de Fibonacci

- Pseudocódigo

```
Algoritmo Serie_Fibonacci
    Definir n,a,b,c,i Como Entero
    Escribir "Ingrese el número de términos n°: ";
    Leer n;
    Si n>2 Entonces
        a<-0;
        b<-1;
        Escribir a," ",b," " Sin Saltar
        Para i<-3 Hasta n Con Paso 1 Hacer
            c<-a+b;
            Escribir c," " Sin Saltar;
            a<-b;
            b<-c;
        Fin Para
    Sino
        Escribir "Error";
    Fin Si
FinAlgoritmo
```

- Código fuente

```
// Ejercicio 11: Generar los primeros términos de la serie de Fibonacci
// Jhoslyn Adriana Pozo Pérez
// URC: 36460

#include <stdio.h>
int main(){
    // Variables para la cantidad de terminos y los valos utilizados en la generación de la serie
    int n,a,b,c,i;

    //Solicitar la cantidad de términos de la serie
    printf("Ingrese la cantidad de numeros para la serie de Fibonacci\n");
    scanf("%d",&n);

    // Validar que la cantidad de términos sea positiva
    while(n<=0){
        printf("Ingrese una cantidad positiva");
        scanf("%d",&n)
    }

    // Inicializar los dos primeros terminos de la serie
    a=0;
    b=1;
```

ESTILO

```
// Mostrar los dos primeros términos
printf("%d %d", a, b);

// Generar y mostrar los términos restantes
for (i=3; i<=n; i++){

    // Cada término se obtiene sumando los dos anteriores
    c = a + b;
    printf(" %d", c)

    // Analizar los valores para calcular el siguiente término
    a = b;
    b = c;
}

return 0;
}
```

Ejercicio 12: Suma de dígitos

• Pseudocódigo

```
Algoritmo Suma_Digitos
    Definir n, digito, suma Como Entero;
    Escribir "Ingrese un número entero positivo";
    Leer n;
    Suma ← 0;
    Mientras n ≠ 0 Hacer
        digito ← n MOD 10;
        suma ← suma + digito;
        n ← TRUNC (n / 10);
    Fin Mientras
    Escribir "La suma de los dígitos es: ", suma
Fin Algoritmo
```

• Código fuente

```
// Ejercicio 12: Calcular la suma de los dígitos de un número entero
// Jhostyn Adriano Poca Pérez
// URC: 36460

#include <stdio.h>
int main () {

    // Variables para almacenar el número, cada dígito extraído y la suma acumulado de
       los dígitos
    int n, d, s;

    // Solicitar un número entero al usuario
    printf("Ingrese un número entero positivo\n");
    scanf("%d", &n);

    // Inicializar el contador de la suma
    s = 0;

    // Extraer y sumar cada dígito del número
    while (n != 0)
```

ESTILO

```
    // Obtener el último digito del número
    d = n % 10;

    // Acumular el digito a la suma total
    s = s + d;

    // Eliminar el último digino del numero
    n = n / 10;
}

// Mostrar el último digito y la suma de todos los digitos
printf("La suma de los digitos es: %d, s);
return 0;
}
```

Ejercicio 7.13 : Número capicua

• Pseudocódigo

```
Algoritmo Número_capicua
    Definir n, aux, digito, invertido Como entero;
    Escribir "Ingrese un número entero positivo";
    Leer n
    Si n <= 0 Entonces
        Escribir "ERROR";
    Sino
        aux <- n
        invertido <- 0
        Mientras n > 0 Hacer
            digito <- n MOD 10;
            invertido <- invertido * 10 + digito;
            n <- TRUNC (n / 10);
        FinMientras
        Si aux = invertido Entonces
            Escribir "El número es capicúa"
        Sino
            Escribir "El número no es capicúa"
        FinSi
    FinSi
FinAlgoritmo
```

• Código fuente

```
// Ejercicio 13: Determinar si un número es capicúa
// Jhostyn Adriano Paca Pérez
// DNI: 36460

#include <stdio.h>

int main(){

    // Variables para almacenar el número original, una copia auxiliar, cada dígito extraído y el número invertido

    int n, aux, d, i;

    //Solicitar un número entero positivo

    printf("Ingrese un número entero positivo\n");

    scanf("%d", &n);

    //Validar que el número sea positivo

    while (n <=0){
```

```
        printf("ERROR: Ingrese un número entero positivo ");
        scanf("%d", &n);
    }

    // Guardar el número original para compararlo posteriormente
    aux = n;

    // Inicializar el número invertido
    i = 0;

    // Invertir los dígitos del número ingresado
    while (n > 0) {
        // Obtener el último dígito del número
        d = n % 10;

        // Agregar el dígito al número invertido
        i = i * 10 + d;

        // Eliminar el último dígito procesado
        n = n / 10;
    }

    // Comparar el número original con el invertido
    if (aux == i) {
        printf("El número %d es capicua", aux);
    }
    else {
        printf("El número %d no es capicua", aux);
    }

    return 0;
}
```

Ejercicio 7.14 Máximo común divisor (Algoritmo de Euclides)

- Pseudocódigo

```
Proceso MCD_Euclides
    Definir a, b, r Como Entero
    Escribir "Ingrese el primer número"
    Leer a;
    Escribir "Ingrese el segundo número: ";
    Leer b;
    Mientras b≠0 Hacer
        r ← a MOD b
        a ← b
        b ← r
    Fin Mientras
    Escribir "El MCD es: " a
Fin Proceso
```

- Código fuente

```
// Ejercicio 14: Calcular el máximo común divisor (MCD) mediante el algoritmo de Euclides
// Yhostyn Adriano Paca Pérez
// UAC: 30460

#include <stdio.h>
int main(){

    // Variables para almacenar los números ingresados y el residuo de los numeros de las divisiones sucesivas
    int a,b,r;

    // Solicitar los dos números
    printf("Ingrese el primer número\n");
    scanf("%d", &a);

    printf("Ingrese el segundo número\n");
    scanf("%d", &b);

    // Validar que ambos números sean enteros positivos
    while(a<=0 || b<=0){
        printf("ERROR: Ingrese números enteros positivos\n");
        scanf("%d", &a);
        scanf("%d", &b);
    }

    // Guardar los valores originales para mostrar al final
    int originalA=a, originalB=b;

    // Aplicar el algoritmo de Euclides
    while(b!=0){
```

```
    // Calcular el residuo de la division
    r = a % b;

    // Actualizar los valores para la siguiente iteración
    a = b;
    b = r;
}

// Cuando b es 0, a contiene el MCD
printf("El MCD entre %d y %d es: %d\n", originalA, originalB, a);
return 0;
}
```

Ejercicio 1.15 : Año bisiesto

• Pseudocódigo

```
Algoritmo Año_Bisiesto
    Definir ano Como entero;
    Definir resultado Como cadena;
    Escribir "Ingrese un año (entero positivo)";
    Leer ano;
    Si ano <= 0 Entonces
        Escribir "ERROR El año debe ser un número positivo";
    Sino
        Si (ano MOD 4 == 0 ) Y (ano MOD 100 ≠ 0) O (ano MOD 400 == 0)
        entonces
            resultado ← "Es bisiesto";
        Sino
            resultado ← "No es bisiesto";
        Fin Si;
        Escribir resultado;
    Fin Si;
Fin Algoritmo
```

• Código fuente

```
// Ejercicio 15 : Determinar si un año es bisiesto
// Jhostyn Adriano Poco Perez
// NRC : 36460

#include<stdio.h>
int main(){

    //Variables para almacenar el año bisiesto o no bisiesto
    int a

    // Solicitar el año al usuario
    printf("Ingrese el año al usuario para ver si es bisiesto o no\n");
    scanf("%d", &a)

    // Validar que el año sea mayor que cero
    while (a <= 0){
        printf("Ingrese un año mayor a 0");
        scanf("%d", &a);
    }

    // Verificar si el año cumple los condiciones para ser bisiesto
    if ((a %400 == 0) || (a %4 == 0 && a %100 != 0)){
        printf("%d no es bisiesto", a)
    }
```

```
    else {
        printf("%d no es bisiesto", a);
    }
    return 0;
}
```

# Ejercicio 1.16: Número perfecto

- Pseudocódigo

```
Algoritmo Número perfecto
    Definir n, i, suma Como Entero
    Escribir "Ingrese un número entero positivo";
    Leer n
    Si n<=0 Entonces
        Escribir "ERROR: El número debe ser entero positivo"
    Sino
        suma <- 0
        Para i <- 1 Hasta n-1 Con Paso 1 Hacer
            Si n MOD i = 0 Entonces
                suma <- suma + i
            FinSi
        FinPara

        Si suma=n Entonces
            Escribir "El número ", n, " es perfecto";
        Sino
            Escribir "El número ", n, " no es perfecto"
        FinSi
    FinSi
Fin Algoritmo
```

- Código fuente

```
// Ejercicio 1.6: Número perfecto
// Jhostyn Adriro Poca Pérez
// UAC: 36460

#include<stdio.h>
int main(){
    int n, i, s;

    // Solicita un número positivo \n
    printf("Ingrese un número positivo \n");
    scanf("%d", &n);

    //Validar que el número sea positivo
    while(n<=0){
        printf("Ser un valor positivo");
        scanf("%d", &n);
    }

    s=0

    //Calcular la suma de los divisores propios
    for(i=1; i<=n-1; i++){
        if(n%i==0){
```

```
            s += i;
        }
    }

    // Determinar si el num es perfecto
    if (s == n) {
        printf("El numero %d es perfecto", n);
    }
    else {
        printf("El numero %d no es perfecto", n);
    }

    return 0;
}
```

Ejercicio 7.17 : Conversión de decimal a binario

- Pseudocódigo

```
Algoritmo Conversion_de_decimal_a_binario
    Definir n, resto Como Entero
    Definir binario Como Cadena
    Escribir "Ingrese un número entero positivo";
    Leer n;
    Si n <= 0 Entonces
        Escribir "ERROR: El número debe ser un entero positivo
    Sino
        binario <- "";
        Mientras n > 0 Hacer
            resto <- n MOD 2
            binario <- ConvertirATexto(resto)+binario
            n <- trunc(n / 2);
        Fin Mientras
        Escribir "Representación binaria: ", binario
    Fin Si
Fin Algoritmo
```

- Código Fuente

```
// Ejercicio 17: Convertir un número decimal a su representación binario
// Jhostyn Adriano Paca Pérez
// URC: 36460

#include<stdio.h>
int main(){

    // Variables para almacenar el número real, los residuos de las divisiones y el número
       binario resultante
    int n, r, binario=0, base=1, original;

    // Solicita el número decimal al usuario
    printf("Ingrese el número para sacar su equivalente en binario\n");
    scanf("%d", &n);

    // Validar que el número original para mostrarlo después
    while(n<0){
        printf("ERROR: Ingrese un número positivo\n");
        scanf("%d", &n);
    }

    // Guardar el número original para mostrarlo al final
    original=n;

    // Obtener el equivalente binario de la división entre 2.
    r=n%2;
```

ESTILO

```
// Construir el número binario mediante divisiones sucesivas entre 2
while (n > 0){

    // Obtener el residuo de la división entre 2
    r = n % 2;

    // Construir el número binario a partir de sus residuos obtenidos
    binario = r * base + binario;

    // Actualizar la posición decimal para el siguiente dígito base
    base = base * 10;

    // Reducir el número para continuar la conversión
    n = n / 2;
}

// Mostrar el resultado de la conversión
printf("El equivalente al número %d en binario es: %ld\n", original, binario);

return 0;
}
```

Ejercicio 1.78 Triangulo de asteriscos

- Pseudocódigo

```
Algoritmo Triangulo de asteriscos
    Definir n, i, j, Como Entero
    Escribir "Ingrese el número de filas: "
    Leer n
    Si n<1 Entonces
        Escribir "Error ", n, "debe ser mayor o igual a 1"
    Sino
        Para r<-1 Hasta n Con Paso 1 Hacer
            Para j<-1 Hasta i Con Paso 1 Hacer
                Escribir "*" Sin Saltar
            Fin Para
            Escribir \n
        FinPara
    FinSi
Fin Algoritmo
```

- Código Fuente

```
// Ejercicio 78 Dibujar una piramide de asteriscos
// Jhostyn Adriano Paco Pérez
// UPC: 36460

#include <studio.h>
int main(){
    // Variables de control para filas, espacios, asteriscos y altura de la piramide
    int i,j,k,n;

    // Solicitar la altura de la pirámide
    printf("Dar un valor pra la piramide\n");
    scanf("%d",&n);

    // Validar que la altura sea mayor o igual a 1
    while(n<1){

        printf("ERROR: Dar un valor mayor o igual a 1");
        scanf("%d",&n);
    }
    // Recorrer cada fila de la piramide
    for(i=1; i<=n; i++){

        // Imprimir los espacios necesarios para centrar la fila
        for(k=1;k<=n-1;k++){
            printf(" ");
        }
```

```
// Imprimir los asteriscos correspondientes a la fila actual
for (j = 1; j <= 2*i - 1; j++) {
    printf("*");
}

// Pasar a la siguiente linea para formar la siguiente fila
printf("\n");
}

return 0;
}
```

Ejercicio 7.19: Segundo mayor de una secuencia

- Pseudocódigo

```
Algoritmo Segundo_mayor_de_una_secuencia
    Definir num, mayor, segundo Como Entero
    Mayor ← 0
    Segundo ← 0
    Escribir "Ingrese numeros enteros positivos (0 para terminar)".
    Repetir
        Leer num
        Si num > 0 Entonces
            Si num > mayor Entonces
                segundo ← mayor
                mayor ← num
            SiNo
                Si (num < mayor) Y (num > segundo) Entonces
                    segundo ← num
                FinSi
            FinSi
        FinSi
    Hasta Que num = 0
    Si mayor = 0 Entonces
        Escribir "No se ingresaron números válidos";
    SiNo
        Escribir "El número mayor es: " mayor;
        Si segundo = 0 Entonces
            Escribir "No existe un segundo valor distinto";
        Sino
            Escribir "El segundo número mayor es: " segundo
        FinSi
    FinSi
FinAlgoritmo
```

- Código Fuente

```
// Ejercicio 19: Encontrar el número mayor y el segundo mayor de la secuencia
// Jhostyn Adriano Poca Pérez
// UFC: 36460

#include <stdio.h>
int main() {

    // Variables para almacenar el número ingresado, el mayor y el segundo valor
    mayor encontrados
    int num, mayor, segundo;

    // Inicializar los valores máximos
    mayor = 0;
    segundo = 0;
```

ESTILO

```
// Solicitar números al usuario hasta ingresar 0
printf("Ingrese n cantidad de numeros (hasta escriba 0 para terminar
        el programa)\n");

do {
    scanf("%d", &num);

    // Considerar unicamente numeros positivos
    if (num > 0) {

        // Si se encuentra un numero mayor, el mayor anterior pasa
        a ser el segundo mayor
        if (num > mayor) {
            segundo = mayor;
            mayor = num;
        }
        else {

            // Actualizar el segundo mayor si el número es menor que el
            anterior mayor
            if (num < mayor && num > segundo) {
                segundo = num;
            }
        }
    }
} while (num != 0);

// Verificar si se ingresaron números válidos
if (mayor == 0) {
    printf("No se ingresaron valores validos\n");
}
else {

    // Mostrar el mayor valor encontrado
    printf("El numero mayor es %d\n", mayor);

    // Verificar si existe un segundo mayor distinto
    if (segundo == 0) {
        printf("No existe un segundo valor distinto\n");
    }
    else {
        printf("El segundo mayor es %d", segundo);
    }
}
return 0;
}
```

Ejercicio 20: Adivina el número

- Pseudo código

```
Algoritmo
    Definir secreto, intento, contador Como Entero
    secreto ← Aleatorio (1, 100);
    contador ← 0;
    Escribir "Adivina el número secreto entre 1 y 100: ";
    Repetir
        Leer intento
        contador ← contador + 1;
        Si intento < secreto Entonces
            Escribir "El número secreto es más alto: "
        SiNo
            Si intento > secreto Entonces
                Escribir "El número secreto es más bajo: ";
            Sino
                Escribir "Acertaste en ", contador, " intentos";
            Fin Si
        Fin Si
    Hasta Que intento = secreto
Fin Algoritmo
```

- Código fuente

```
// Ejercicio 20: Juego para adivinar un número aleatorio entre 1 y 100
// Jhostyn Adriano Paca Pérez
// UAC: 36460

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {

    // Variables para almacenar el número secreto, el intento del usuario y la cantidad de intentos realizados
    int secreto, intento, contador = 0;

    // Inicializar la semilla para generar números almacenados diferentes
    srand(time(NULL));

    // Generar un número aleatorio entre 1 y 100
    secreto = rand() % 100 + 1;

    // Mostrar las instrucciones del juego
    printf("He pensado un número entre 1 y 100.\n");
    printf("Intenta adivinarlo.\n");
```

```
// Repetir hasta que el usuario adivine el número secreto
do {
    scanf("%d", &intento);
    // Contabilizar cada intento realizado
    contador++;
    // Indicar si el número buscado es mayor o menor
    if (intento < secreto) {
        printf("¡Más alto!\n");
    } else if (intento > secreto) {
        printf("¡Más bajo!\n");
    } else {
        // El usuario ha encontrado el número correcto
        printf("¡Acertaste!\n");
    }
} while (intento != secreto);
// Mostrar el número total de intentos utilizados
printf("Usaste %d intentos.\n", contador);
return 0;
}
```